3RD SEMESTER MCA

MCA 5141 – MACHINE LEARNING LAB

FINAL PROJECT REPORT

A Comparative Study of Machine Learning Models for Indian Road Accident Severity Classification

Machine Learning Classification and Streamlit Demonstration

Project Details	Information
Project Title	Indian Road Accident Severity Classification
Subject	MCA 5141 – Machine Learning Lab
Platform	Python, Jupyter Notebook, Streamlit
Dataset	indian_roads_dataset.xlsx
GitHub Repository	Shubhankamin/python-project

Team Members

Member	Name and Registration Number
Member 1	Shubhank Amin (251090700127)
Member 2	Ullas BH (251090700125)

School of Computer Engineering
Manipal Institute of Technology, Manipal, India

Contents

1. Abstract
2. Problem Statement and Objectives
3. Dataset Metadata
4. Exploratory Data Analysis
5. Preprocessing Pipeline and Methodology
6. Machine Learning Models
7. Model Evaluation and Comparison
8. Model Selection and Interpretation
9. Application Development and Deployment
10. Responsible Use of AI Tools
11. Limitations and Ethical Considerations
12. Conclusion and Future Scope

1. Abstract

In this project, we compared six machine-learning models to predict road accident severity as minor, major, or fatal. We used the supplied dataset, which contains 20,000 records and 24 original columns, and evaluated the models with an 80:20 stratified train-test split. Decision Tree gave the best Macro F1-score (0.3249), while Gradient Boosting had the highest accuracy (0.55025). However, Gradient Boosting did not identify any fatal cases in the recorded test results. This showed us why accuracy alone is not enough for an imbalanced dataset. We also built a Streamlit demonstration, but the results are not suitable for real-world safety decisions.

2. Problem Statement and Objectives

2.1 Problem Statement

Our aim was to find out how well machine-learning models could classify the supplied accident records into three groups: minor, major, and fatal. The number of records in each group is different, so a model can appear accurate while performing poorly on the less common fatal class. We compared accuracy with Macro F1 and checked the fatal-class results before choosing a model.

2.2 Objectives

- Inspect the dataset structure, feature types, missing values, and target distribution.
- Perform exploratory analysis and prepare a preprocessing pipeline for mixed feature types.
- Train and evaluate six classification algorithms on a stratified train-test split.
- Compare accuracy and Macro F1-score, with attention to class-wise performance.
- Select a model using a stated evaluation criterion and explain the trade-offs.
- Review and describe the Streamlit demonstration and its limitations.

3. Dataset Metadata

We used the file indian_roads_dataset.xlsx for this project. It contains 20,000 rows and 24 columns before preprocessing. The target column is accident_severity, with three labels: minor, major, and fatal. The project files do not state where the dataset came from or how it was collected, so we cannot confirm that it represents real-world accident records.

Metadata item	Observed value
Rows × original columns	20,000 × 24
Target variable	accident_severity
Target classes	minor, major, fatal
Training/test split	80% / 20%, stratified, random_state=42
Training rows	16,000
Test rows	4,000
Missingness requiring attention	festival: 19,885 missing values
Dataset provenance	Not documented in the supplied project files

3.1 Original columns

The original columns are: accident_id, city, state, latitude, longitude, date, time, hour, day_of_week, is_weekend, road_type, lanes, traffic_signal, weather, visibility, temperature, traffic_density, cause, accident_severity, vehicles_involved, casualties, is_peak_hour, festival, and risk_score.

3.2 Target distribution

Severity class	Records	Share
minor	11,025	55.12%
major	5,988	29.94%
fatal	2,987	14.94%

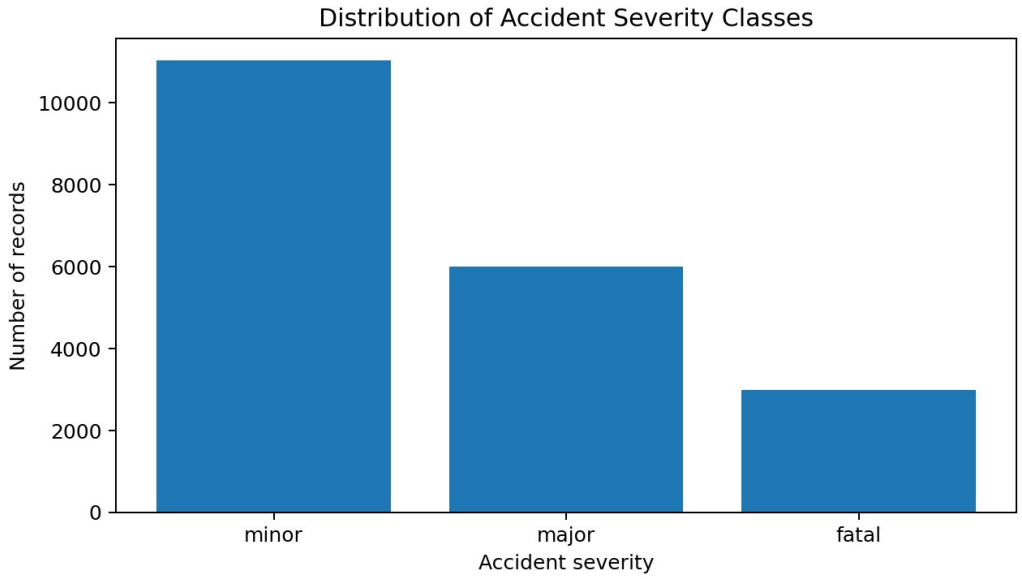


Figure 1. Target class distribution calculated from the supplied dataset.

4. Exploratory Data Analysis

The dataset has more minor accidents than major or fatal accidents. Minor records account for 11,025 cases (55.12%), major records account for 5,988 (29.94%), and fatal records account for 2,987 (14.94%). This imbalance matters because a model may perform well on the largest class and still miss many fatal cases.

The original column festival contains 19,885 missing entries. The notebook drops this column before model training rather than imputing it. The notebook also drops accident_id, state, date, time, cause, vehicles_involved, casualties, and risk_score. The code comments do not establish that all removed columns are definitively leakage; excluding cause, vehicles_involved, casualties, and risk_score is a cautious modelling decision because these may describe an accident after it occurred or encode information unavailable at prediction time.

The notebook includes plots for the severity classes and several features, including city, road type, weather, traffic density, accident hour, and temperature by severity. Since the project files do not explain the dataset's source, we describe these as patterns in the supplied data rather than confirmed trends for all road accidents.

5. Preprocessing Pipeline and Methodology

5.1 Feature engineering and column selection

For feature preparation, the notebook converts the date field to a date format, extracts the year and month, and converts the hour into hour_sin and hour_cos. These two features represent time as a cycle, so hours near midnight are treated as close to each other. The notebook then removes the columns listed in the preprocessing section.

5.2 Preprocessing

Feature group	Columns / approach
Numeric	latitude, longitude, hour, is_weekend, lanes, traffic_signal, temperature, is_peak_hour, year, month, hour_sin, hour_cos; median imputation and StandardScaler
Nominal categorical	city, day_of_week, road_type, weather; most-frequent imputation and OneHotEncoder(handle_unknown='ignore')
Ordinal categorical	visibility and traffic_density; most-frequent imputation and OrdinalEncoder with low < medium < high
Combined preprocessing	ColumnTransformer combined with each classifier in a scikit-learn Pipeline

5.3 Split and experimental setup

We split the data into 80% for training and 20% for testing. The split is stratified, which keeps the class proportions roughly similar in both sets. With random_state=42, the split contains 16,000 training records and 4,000 test records. We used the same split setup to compare the six classifiers fairly.

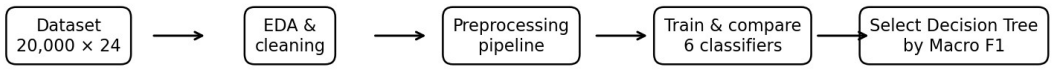


Figure 2. Workflow reconstructed from the supplied notebook.

6. Machine Learning Models

Model	Brief description	Ensemble?
Logistic Regression	Linear classifier that estimates class probabilities.	No
K-Nearest Neighbors	Predicts using labels of nearby training examples.	No
Decision Tree	Learns a sequence of feature-based decision rules.	No
Support Vector Machine	Finds decision boundaries separating classes.	No
Random Forest	Combines predictions from multiple decision trees.	Yes
Gradient Boosting	Builds trees sequentially to improve previous errors.	Yes

The notebook configuration uses class_weight='balanced' for Logistic Regression, Decision Tree, SVM, and Random Forest; KNN uses five neighbors, Random Forest uses 100 trees, and Gradient Boosting uses random_state=42. The recorded comparison table is the source of the metrics reported below.

7. Model Evaluation and Comparison

7.1 Metrics

Accuracy tells us what fraction of all predictions were correct. Macro F1 calculates an F1-score for each class and then gives the classes equal weight. Precision measures how often a predicted class is correct, while recall measures how many examples of a class the model finds. We included fatal-class recall because missing fatal cases is an important weakness in this project.

Model	Accuracy	Macro F1	Fatal recall
Decision Tree	0.4060	0.3249	0.139
KNN	0.4165	0.3145	0.097
Logistic Regression	0.3267	0.3113	0.355
Random Forest	0.4567	0.3084	0.035
SVM	0.3180	0.2968	0.313
Gradient Boosting	0.5503	0.2404	0.000

Decision Tree had the highest Macro F1-score in our recorded comparison (0.3249). Gradient Boosting had the highest accuracy (0.55025), but its fatal-class recall was zero. The majority-class baseline also had an accuracy of about 0.551, which is a reminder that accuracy can be misleading when one class is more common than the others. Figure 3 compares the Macro F1-scores; Figure 4 gives the class-wise errors for Decision Tree. The plots should be interpreted with the evaluation table, particularly the low fatal-class recall.

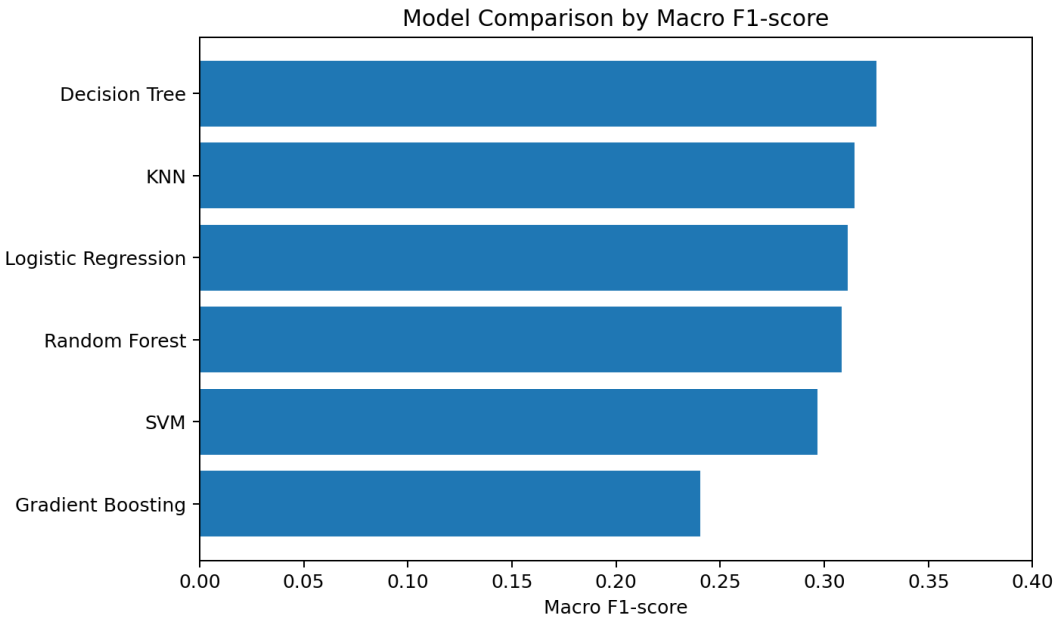


Figure 3. Macro F1 comparison using the project's recorded CSV results.

7.2 Decision Tree class-wise results

Actual class	Precision	Recall	F1-score	Support
fatal	0.13	0.14	0.14	597
major	0.29	0.30	0.29	1,198
minor	0.55	0.54	0.54	2,205

The Decision Tree detected only about 14% of the fatal examples in the test set. In other words, it missed most of them. This is a serious limitation, even though Decision Tree had the best Macro F1-score among the six models. It should not be treated as a reliable tool for assessing actual accident severity.

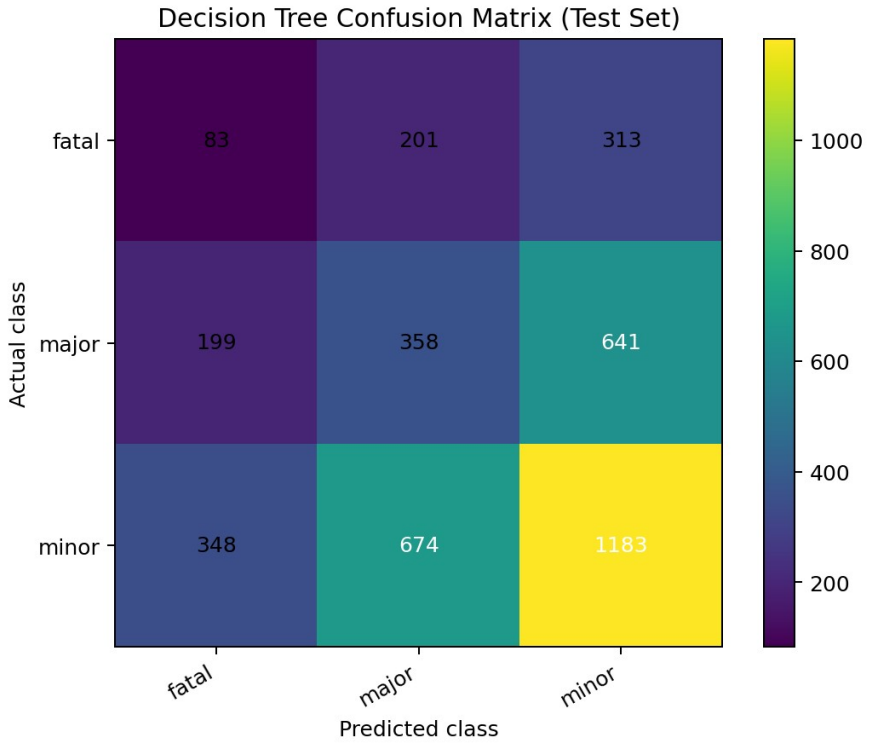


Figure 4. Decision Tree confusion matrix reproduced from the executed notebook output.

8. Model Selection and Interpretation

We selected Decision Tree because it had the highest Macro F1-score among the six models we tested. Its accuracy was 0.4060 and its Macro F1-score was 0.3249. We did not choose it because it was highly accurate overall or because it reliably detected fatal accidents. Its fatal-class recall was only about 0.14, so the results leave plenty of room for improvement.

The results show that none of the tested models is strong enough to support real-world decisions. A useful next step would be to check the data source, investigate the errors for each class, tune the models carefully, and test them on a separate dataset. The model would also need to be evaluated on trustworthy real-world data before any practical use could be considered.

9. Application Development and Deployment

The project includes a Streamlit app that lets a user enter feature values and view a predicted severity class. The app is intended as a simple demonstration of the prediction workflow. It prepares the input features and trains a Decision Tree when the app starts.

One difference to note is that app.py trains its own Decision Tree when it starts; it does not load the exact saved model from the notebook's evaluation run. Therefore, the results reported in this document should not be assumed to match every prediction from the deployed app. This is something we would improve by saving and reusing the same fitted pipeline.

The app displays a warning that it is for educational use only. The deployment link provided for the project is https://road-accident-severity-predictor-szvq2rgehvz7hi5ba6sxfd.streamlit.app/. We have recorded the link here, but this report does not confirm that the app is always online.

10. Responsible Use of AI Tools

We used ChatGPT for targeted help when we needed an explanation, wanted to organise the report, or needed to check whether our descriptions matched the project results. We still needed to run the notebook, inspect the results, read the app code, and decide what the results actually showed. AI was a support tool; it did not replace our responsibility to understand the project.

10.1 Declaration of AI Usage with Prompts

Scope and accuracy note: This is a reconstructed, representative log of AI assistance based on the project materials and conversation details available while preparing this report. It is not a complete export of every AI interaction from the entire project. The prompt wording in the table is presented as a summary unless explicitly shown as a quotation. Before submission, both team members must check the entries against the tools they actually used, add any missing tools or important prompts, and remove any entry that does not reflect their own work. No tool or prompt should be listed merely because it would be useful to include.

The table below summarises the main requests we made while working on the project and preparing the report. The wording is reconstructed as concise prompt summaries, not presented as a verbatim chat transcript. Before submission, we should remove any entry that does not match the help we actually used and add any missing tool or prompt.

Tool / task	Key prompt or request summary	Output used	Verification / change made
ChatGPT – Phase 1 planning	“For Phase 1 of our road accident severity project, help us define the problem and objectives, document the dataset metadata, plan EDA, describe the preprocessing approach, and shortlist six suitable classifiers including two ensemble models.”	Used the guidance to organise the initial project plan and identify the items to document.	Checked the plan against the Phase 1 instructions and kept the dataset details and planned steps consistent with our project files.
ChatGPT – checking the dataset	“Help me inspect the road-accident dataset: what should I check about its shape, columns, target, and missing values?”	Used the explanation to describe the dataset inspection.	Checked the row count, column count, target labels, and missing-value details in the notebook.

Tool / task	Key prompt or request summary	Output used	Verification / change made
ChatGPT – understanding class imbalance	“Our target has minor, major, and fatal classes in different proportions. Explain class imbalance and why it affects evaluation.”	Used the explanation to discuss the different numbers of minor, major, and fatal records.	Compared the explanation with the actual class counts and percentages.
ChatGPT – understanding preprocessing	“Explain me all the preprocessing steps used in this notebook and explain what each one is for.”	Used the explanation to help describe the preprocessing steps.	Kept only the preprocessing methods that appear in our notebook.
ChatGPT – Phase 2 model implementation	“For Phase 2, help me understand how to train and compare our six selected classifiers, including Random Forest and Gradient Boosting, using the prepared dataset and an 80:20 stratified split.”	Used the explanation to organise the model implementation and evaluation discussion.	Verified the model names, split settings, and reported metrics against the notebook outputs; did not add results that were not recorded.
ChatGPT – understanding the six models	“Give a short, simple and beginner-friendly explanation of the six classifiers we evaluated, including which two are ensemble methods.”	Used the explanations to write short descriptions of the tested models.	Removed any model that was not part of our recorded comparison.
ChatGPT – comparing model results	“Using the results, compare the models by accuracy and Macro F1. Do not add metrics that are not in the results.”	Used the explanation to describe how the model rankings differ.	Checked all reported values against the saved results before using them.
ChatGPT – understanding fatal recall	“Explain fatal-class recall using the Decision Tree result, and help me describe it in simple words properly.”	Used the explanation to describe the model's difficulty identifying fatal cases.	Kept the limitation clear and did not describe the model as safe for real-world use.
ChatGPT – confusion matrix	“How to interpret the cells in this Decision Tree confusion matrix? Explain correct and incorrect predictions by class.”	Used the explanation to describe the difference between actual and predicted classes.	Checked the class labels and values against the notebook output.
ChatGPT – Phase 3 requirements and deployment	“For Phase 3, check the lab manual requirements for model deployment, application demonstration, complete code submission, and final report sections. What should we verify before submission?”	Used the checklist to review the final report and deployment description.	Compared the checklist with the lab manual and project files; retained limitations where the app and notebook differ.
ChatGPT – checking draft claims	“Review these draft statements against the notebook results and app.py. Flag any claim that the project files do not support.”	Used the review to identify statements needing evidence.	Left out claims that could not be supported by the project files.
ChatGPT – explaining the Streamlit app	“Trace the prediction flow in our Streamlit app.py and explain what happens when a user submits input.”	Used the explanation to describe the web demonstration.	Checked the explanation against the supplied source code and avoided claiming that the app loads a saved model when it does not.

Tool / task	Key prompt or request summary	Output used	Verification / change made
ChatGPT – writing the AI-use section	"Help arrange our AI-use declaration, critical review, learning reflection, and design-ownership explanation under the lab manual headings."	Used the structure to cover the lab manual's responsible-AI requirements.	Edited the wording so it describes AI as support, not as the person who completed the project.

10.2 Critical Evaluation of AI Output

We did not use every suggestion from AI. For example, an earlier draft reported XGBoost with 86.4% accuracy and described methods or features that we could not find in our project files. We removed those claims rather than presenting them as results. We also checked the recorded model scores and found that Gradient Boosting had the highest accuracy but zero fatal-class recall, while Decision Tree had the highest Macro F1-score.

We checked app.py before describing the deployment. The app trains a Decision Tree at startup instead of loading the exact fitted model evaluated in the notebook. We included this difference as a limitation rather than claiming that the deployed app uses the same saved model. Checking these details helped us avoid overstating what the project can do.

10.3 Reflection on Learning

Reflection on learning means briefly explaining what we understood or improved while doing the project, rather than simply saying that the project was completed. We learned that missing values and unequal class sizes affect how data should be prepared and how model results should be judged. Comparing the six models also showed us that the highest accuracy does not always mean the model works well for every class. We also learned to check AI suggestions against the notebook and app code before using them in the report.

Shubhank Amin: While working on this project, I learned to look beyond the first few rows of a dataset and check its size, columns, missing values, and target-class distribution before training a model. The difference between accuracy and Macro F1 helped me understand why a model can score well overall but still perform poorly on a smaller class. I also learned that the numbers in a report must be checked against the notebook outputs instead of being copied from a draft. I should be able to explain the dataset checks and the model-comparison results during the viva.

Ullas BH: This project helped me connect the data-preparation steps with model evaluation and the Streamlit prediction workflow. I learned that the model with the highest accuracy is not automatically the most useful choice, especially when fatal cases are important. The Decision Tree had the highest Macro F1 in our comparison, but its fatal-class recall was low, so the result needs to be described honestly. I also learned to check the app code separately from the notebook because the app trains its own model at startup. I should be ready to explain these limitations and the prediction flow during the viva.

10.4 Problem Analysis and Design Ownership

We framed the project as a three-class classification task, with accident_severity as the target and minor, major, and fatal as the labels. The design follows a sequence: inspect the dataset and target distribution; remove or transform columns as implemented in the notebook; prepare the input features; split the data using stratification; train six classifiers; compare their metrics; and demonstrate a prediction through Streamlit. Figure 2 shows this workflow. These steps describe the design used in the project rather than a claim that the system is ready for real-world accident decisions.

Our design choices were based on the recorded experiment: we used the accident_severity target with minor, major, and fatal classes, prepared the features as implemented in the notebook, and used a stratified 80:20 train-test split. We compared Logistic Regression, KNN, Decision Tree, SVM, Random Forest, and Gradient Boosting. Decision Tree was selected because it achieved the highest recorded Macro F1-score (0.3249), not because it was strong at detecting fatal cases. Its fatal-class recall was about 0.14, so many fatal cases were missed. We also noted that the Streamlit app trains its own Decision Tree instead of loading the exact fitted model evaluated in the notebook.

We used an 80:20 stratified train-test split with random_state=42 so that the test set retained a similar class distribution. The six models were Logistic Regression, KNN, Decision Tree, SVM, Random Forest, and Gradient Boosting; Random Forest and Gradient Boosting are the two ensemble methods. Accuracy gives an overall view, while Macro F1 gives equal importance to each class. Fatal-class recall was also considered because missing fatal cases is an important limitation. Decision Tree was selected by Macro F1 (0.3249), not by accuracy; its fatal recall was only about 0.14. Gradient Boosting had the highest accuracy (0.55025) but zero fatal recall in the recorded test results.

AI helped us explain some concepts and organise the report, but the final descriptions need to match our actual work. We are responsible for understanding the preprocessing steps, reading the model results, explaining why Decision Tree was selected, and describing the app's limitations. Each team member should be ready to explain the parts they personally worked on.

Shubhank Amin (251090700127): I contributed to understanding the road accident dataset, reviewing its columns and target classes, and examining the model evaluation results. I learned how missing values and class imbalance affect the classification task. I also helped compare the models using accuracy and Macro F1-score to understand their strengths and limitations.

Ullas BH (251090700125): I contributed to reviewing the preprocessing and model workflow and understanding how the trained classification process connects to the Streamlit application. I examined the model comparison, especially the low fatal-class recall of the selected Decision Tree model. I also reviewed the application's limitations and learned why the deployed application should not be treated as a validated real-world road safety system.

Design component	Decision to explain during viva
Target and task	Three classes: minor, major, fatal; target is accident_severity.
Data preparation	Inspect missingness and remove columns as implemented in the notebook.
Feature engineering	Create year, month, hour_sin, and hour_cos from temporal fields.
Evaluation design	Use a stratified 80:20 split with random_state=42.
Model comparison	Compare six classifiers using accuracy, Macro F1, and fatal recall.
Selection and deployment	Select Decision Tree by Macro F1 and explain app/notebook differences.

11. Limitations and Ethical Considerations

- Dataset provenance and collection methodology are undocumented in the supplied files; the dataset may be synthetic.
- The target classes are imbalanced, and fatal-class recall is low for the selected Decision Tree.
- Gradient Boosting achieves higher accuracy but has zero fatal-class recall in the recorded run.

- The recorded scores are not evidence of generalization to actual road incidents or geographic regions.
- The Streamlit app retrains its own Decision Tree at startup rather than loading the exact saved evaluated model.
- Predictions must not be used for emergency response, policing, traffic management, insurance, or other real-world safety decisions.

12. Conclusion and Future Scope

We compared six classifiers using the supplied road accident dataset. Decision Tree achieved the highest Macro F1-score (0.3249), while Gradient Boosting achieved the highest accuracy (0.55025) but did not identify fatal cases in the recorded test run. The comparison showed us that a single accuracy value does not describe how well a model handles every class. The Streamlit app demonstrates the prediction process, but the low fatal recall, undocumented data source, and difference between notebook and app training mean that the project is not validated for real-world use.

For future work, we would first document the dataset source and check its quality. We would then investigate class-wise errors, tune the models, improve the consistency between notebook training and the deployed app, and test the final pipeline on independent data. Only after careful validation could its practical usefulness be assessed.